

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatics

Mihail Pancenco FAF-243

Report

Laboratory work No. 9

Operating Systems

Watching System Calls with strace

Supervisors:
Dumitraşcu Marius, University assistant
Reiţman Patricia, University assistant
DISA, FCIM, UTM

Chişinău – 2026

Table of Contents

1 Introduction **3**

2 Tasks and results **3**

2.1 Observe ls with strace 3

2.2 Write and read a file through the kernel 4

2.3 Count one kind of call 5

3 Conclusions **5**

A Submitted source code **6**

1 Introduction

This laboratory observes the boundary between Linux programs and the kernel. The commands ran as `misa` in Ubuntu 26.04.1 LTS (AArch64) on VirtualBox on 9 November 2026, using strace 6.19 and GCC 15.2.0. The figures show the actual terminal sessions.

2 Tasks and results

2.1 Observe ls with strace

The test directory initially contained two small files, `alpha.txt` and `beta.txt`. The `-f` option follows child processes; `-e trace=...` limits the displayed system calls. The `-o` option saves the trace separately from the listing.

```
misa@UbuntuVM:sample$ strace -f -e trace=openat,read,write,close -o ../ls-filtered.strace ls -l
total 8
-rw-rw---- 1 misa misa 6 Nov  9 14:41 alpha.txt
-rw-rw---- 1 misa misa 5 Nov  9 14:41 beta.txt
misa@UbuntuVM:sample$ tail -12 ../ls-filtered.strace
6062  close(4)                          = 0
6062  openat(AT_FDCWD, "/usr/share/zoneinfo/Etc", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 4
6062  close(4)                          = 0
6062  openat(AT_FDCWD, "/usr/share/zoneinfo/Australia", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 4
6062  close(4)                          = 0
6062  openat(AT_FDCWD, "/usr/share/zoneinfo/Etc/UTC", O_RDONLY|O_CLOEXEC) = 4
6062  read(4, "TZif2\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..., 114) = 114
6062  read(4, "", 32)                   = 0
6062  close(4)                          = 0
6062  close(3)                          = 0
6062  write(1, "total 8\n-rw-rw---- 1 misa misa 6"..., 103) = 103
6062  +++ exited with 0 +++
misa@UbuntuVM:sample$
```

Figure 2.1 - Filtered ls trace and the write to standard output.

The listing was printed with `write(1, ..., 103) = 103`. Descriptor **1** is standard output, and all 103 bytes from that call were written. The filtered trace recorded **85** calls, including file access and setup activity such as reading timezone data. The unfiltered run recorded **264** syscall entries for the same two-file directory.

2.2 Write and read a file through the kernel

The program opens `note.txt` with create, write-only and truncate flags; writes the message; closes the file; and opens it again for reading. It checks each operation and verifies the bytes before printing them. `printf` is used only for terminal messages; the file itself is accessed with `open`, `write`, `read` and `close`.

```
misa@UbuntuVM:sample$ gcc -Wall -Wextra -Werror -o syscalls ../syscalls.c
misa@UbuntuVM:sample$ ./syscalls
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
misa@UbuntuVM:sample$ wc -c note.txt
14 note.txt
misa@UbuntuVM:sample$
```

Figure 2.2 - Compilation and verification of the 14-byte file.

```
misa@UbuntuVM:sample$ strace -e trace=openat,write,read,close -o ../syscalls.strace ./syscalls
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
misa@UbuntuVM:sample$ sed -n '/"note.txt"/,$p' ../syscalls.strace
openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(1, "open returned descriptor 3\n", 27) = 27
write(3, "hello, kernel\n", 14)         = 14
write(1, "write sent 14 bytes\n", 20)   = 20
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_RDONLY)  = 3
read(3, "hello, kernel\n", 63)          = 14
write(1, "read got 14 bytes: hello, kernel"..., 33) = 33
close(3)                                = 0
+++ exited with 0 +++
misa@UbuntuVM:sample$
```

Figure 2.3 - File I/O on descriptor 3 and terminal diagnostics on descriptor 1.

Descriptors **0**, **1** and **2** are inherited as standard input, standard output and standard error. The first `openat` therefore returns the lowest unused number, **3**. After it is closed, the read-only open can reuse that number. On this system the C library implements the program's `open` calls with `openat`; `AT_FDCWD` resolves the relative filename against the current working directory. The message has 14 bytes including the newline, without its C string terminator.

2.3 Count one kind of call

An unfiltered `strace -f` run on the same directory saved the complete listing of calls. Counting lines that begin with a PID and a call name gave **264** syscall entries. A separate `strace -f -c -e trace=openat` run counted **31 openat calls**, including **one failed call**. The latter summary used redirected standard output; it counts one syscall type rather than all calls.

```
misa@UbuntuVM:sample$ strace -f -o ../ls-full.strace ls -l
total 8
-rw-rw---- 1 misa misa 6 Nov  9 14:41 alpha.txt
-rw-rw---- 1 misa misa 5 Nov  9 14:41 beta.txt
misa@UbuntuVM:sample$ grep -Ec '^[0-9]+ +[[:alnum:]_]+\(' ../ls-full.strace
264
misa@UbuntuVM:sample$ strace -f -c -e trace=openat ls -l >/dev/null
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
100.00    0.000949          30        31         1 openat
------ ----------- ----------- --------- --------- ----------------
100.00    0.000949          30        31         1 total
misa@UbuntuVM:sample$ strace -f -c -e trace=openat -o ../openat-summary.txt ls -l >/dev/null
misa@UbuntuVM:sample$
```

Figure 2.4 - Total syscall entries and the openat-only summary.

The count includes setup work and failed attempts, so it is an observation for this version of `ls`, this directory and this run. The submitted raw traces show the difference between filtering, counting, and tracing every syscall.

3 Conclusions

The trace distinguished the application's file operations from its terminal output and library startup. A short `ls` invocation made hundreds of calls; the sample program's own file activity was easy to identify by its filename, descriptor and byte counts.

Bibliography

[1] Operating Systems, FCIM / FAF, Technical University of Moldova. Laboratory work No. 9: *Watching System Calls with strace*. Course handout, 2026–2027.

[2] Ubuntu manual pages for `strace(1)`, `open(2)`, `read(2)`, `write(2)` and `close(2)`.

A Submitted source code

```c
/* Build: gcc -Wall -Wextra -Werror -o syscalls syscalls.c */
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

int
main(void)
{
    const char msg[] = "hello, kernel\n";
    const size_t length = sizeof(msg) - 1;
    char buf[64];
    ssize_t n;

    int fd = open("note.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd < 0) { perror("open for writing"); return 1; }
    printf("open returned descriptor %d\n", fd);

    do { n = write(fd, msg, length); } while (n < 0 && errno == EINTR);
    if (n < 0) { perror("write"); close(fd); return 1; }
    if ((size_t)n != length) {
        fputs("short write\n", stderr);
        close(fd);
        return 1;
    }
    printf("write sent %zd bytes\n", n);
    if (close(fd) < 0) { perror("close after writing"); return 1; }

    fd = open("note.txt", O_RDONLY);
    if (fd < 0) { perror("open for reading"); return 1; }
    do { n = read(fd, buf, sizeof(buf) - 1); } while (n < 0 && errno == EINTR);
    if (n < 0) { perror("read"); close(fd); return 1; }
    if ((size_t)n != length || memcmp(buf, msg, length) != 0) {
        fputs("unexpected file contents\n", stderr);
        close(fd);
        return 1;
    }
    buf[n] = '\0';
    printf("read got %zd bytes: %s", n, buf);
    if (close(fd) < 0) { perror("close after reading"); return 1; }
    return 0;
}
```